

CS 527 Research Project Report: Group-5

Alex Krysiuk, Shuyu Cai
University of Illinois at Urbana-Champaign
[krysiuk2,shuyuc4]@illinois.edu

Abstract

The reliability of autonomous software engineering agents is a prerequisite for their adoption in real-world development workflows. While Large Language Models have demonstrated the ability to resolve complex GitHub issues, their probabilistic nature introduces a new class of flakiness within the software lifecycle. Current evaluation benchmarks primarily focus on static pass rates, which fails to account for the non-deterministic variance in agent reasoning and code generation. This study presents a framework for Agent Flakiness Analysis by treating the agentic process itself as the test subject. We repeatedly execute the SWE-Agent on previously resolved issues to isolate sources of instability, categorizing failures into reasoning divergence and output inconsistency. In doing so, our approach bridges the gap between traditional regression testing and AI-driven development, offering metrics that distinguish robust engineering logic from stochastic success.

1 Artifact Information

Link to the GitHub Repository: [put the link here]

2 Introduction

Determinism has been and continues to be a foundation of software engineering. Given identical inputs and environments, a program is expected to produce identical outputs. However, agentic systems driven by LLMs are inherently probabilistic. If an agent solves a GitHub issue once but fails on subsequent attempts, it introduces a new class of "flakiness" into the development pipeline. Understanding this instability is essential, as an agent that non-deterministically modifies codebases poses significant risks to regression testing and system stability.

Current benchmarks for evaluating agents like SWE-Agent and Devin are largely oriented around maximizing overall solve rates on standardized datasets. Techniques such as Chain-of-Thought (CoT) prompting and Self-Correction have become standard practice for boosting reasoning quality. While these approaches improve the probability of arriving at a correct solution, they leave an important dimension largely unexamined: the variance inherent in the solution process itself. Existing literature on "flaky tests," as explored by Luo et al. and others, tends to trace non-determinism back to the code being tested, citing culprits like concurrency issues and asynchronous timing. What remains underexplored, however, is the non-determinism introduced by the entity writing the fix — whether that is a human developer or an AI agent.

There is a lack of empirical analysis regarding the stability of agentic success. Current benchmarks do not distinguish between an agent that genuinely understands a codebase and one that arrives at a correct patch through a "lucky" random seed or a specific, non-reproducible hallucination sequence. Consequently, there are no established metrics to quantify "Agent Flakiness" or to determine if

a failure is caused by the generated code being flaky or the agent's reasoning.

This project proposes a framework for Agent Flakiness Analysis that treats the agentic workflow itself as the test subject. Subjecting a set of previously solved issues to iterative execution allows us to isolate the sources of variance and categorize failures into two types: reasoning divergence, where differing thought processes lead to incorrect actions, and output instability, where correct reasoning nevertheless produces flaky patches. Together, these concepts bridge the gap between LLM evaluation and regression testing principles.

The experiment utilizes the SWE-Agent framework on a subset of verified successful trajectories from the course Mini-Project. We will select five distinct GitHub issues that the agent has previously solved and execute the agent twenty times against each issue. The study will capture the trajectory logs, the final patch, and the test execution results for every run to calculate flakiness metrics. One paragraph about the results of experiments

A list of contributions your technique/paper makes.

3 Proposed Technique

3.1 Approach Overview

Our goal is to study the instability of software engineering agents when solving the same bug multiple times. We design a pipeline that repeatedly runs an agent on the same bug report and analyzes the differences in behavior across runs.

The pipeline starts with a bug report that was previously solved successfully by the agent. The agent runner executes the task multiple times under the same environment and configuration. For each run, we collect the full trajectory, including reasoning steps, tool calls, generated patches, and test results.

After collecting data from all runs, we perform outcome analysis to identify unstable behavior. We compare success and failure cases, differences in generated patches, and variations in reasoning paths. We also compute flakiness-related metrics to quantify instability.

As an extension, we introduce a validator agent that compares the final patches across runs. This validator checks whether successful patches are consistent or significantly different, helping us distinguish between stable solutions and random success.

3.2 Approach Details

Describe the details of each component in this subsection

4 Experimental Evaluation

We design experiments to answer the following research questions:

- How often does the agent produce inconsistent outcomes for the same bug report?
- How different are the generated patches across runs?

- How much variation exists in the agent’s reasoning trajectories?

To answer these questions, we repeatedly execute the agent under identical settings and analyze the collected data.

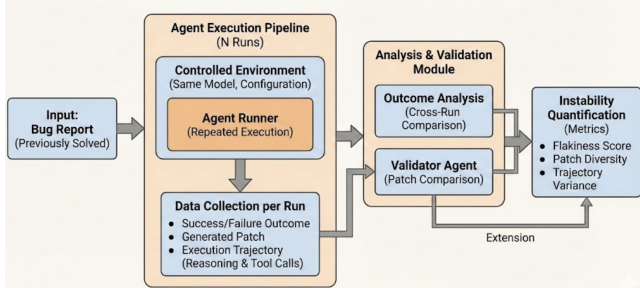


Figure 1: Framework Overview Figure

4.1 Evaluation Setting

All experiments are conducted in a controlled environment. We use the same model, prompts, tools, and configuration for every run. For each selected bug report, the agent is executed 20 times.

For each run, we record the following information:

- Success or failure outcome
- Generated patch
- Execution trajectory (reasoning steps and tool calls)
- Execution time

We evaluate instability using the following metrics:

- **Flakiness Score:** Measures how often outcomes differ across runs.
- **Patch Diversity:** Measures differences among generated patches.
- **Time to Success:** Measures how long it takes to obtain a correct solution.
- **Trajectory Variance:** Measures differences in reasoning paths across runs.

These metrics allow us to quantify the reliability of the agent when solving the same task repeatedly.

4.2 Evaluation Results

Present the results of individual research questions in this subsection. Use tables/figures to present the results. You need to discuss your observations and speculate the reasons for observations.

5 Relate Work

- (1) Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014). Association for Computing Machinery, New York, NY, USA, 643–653. <https://doi.org/10.1145/2635868.2635920>

Establishes the foundational empirical taxonomy of flaky tests by categorizing root causes like Async Wait, Concurrency, and Order Dependency. This study provides the framework used here to determine if agent failures result from traditional code instability or logic errors.

- (2) Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2025. An Empirical Study of the Non-Determinism of ChatGPT in Code Generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (January 2025), 1–28. DOI:<https://doi.org/10.1145/3697010>

Explores how temperature and sampling settings influence the consistency of model outputs, providing the theoretical basis for agent reasoning divergence where identical inputs produce different cognitive paths and ultimately different patches.

- (3) John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. Retrieved from <https://arxiv.org/abs/2405.15793>

Describes the agentic system and the Agent-Computer Interface used for environment interaction, providing both the baseline performance data and the Thought-Action-Observation trajectory structure that this work investigates for non-deterministic behavior.

- (4) Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? Retrieved from <https://arxiv.org/abs/2310.06770>

Introduces the evaluation framework for testing language models on real GitHub issues. It highlights the difficulty of multi-file coordination and establishes the benchmarks used to identify the successful trajectories analyzed in this project.